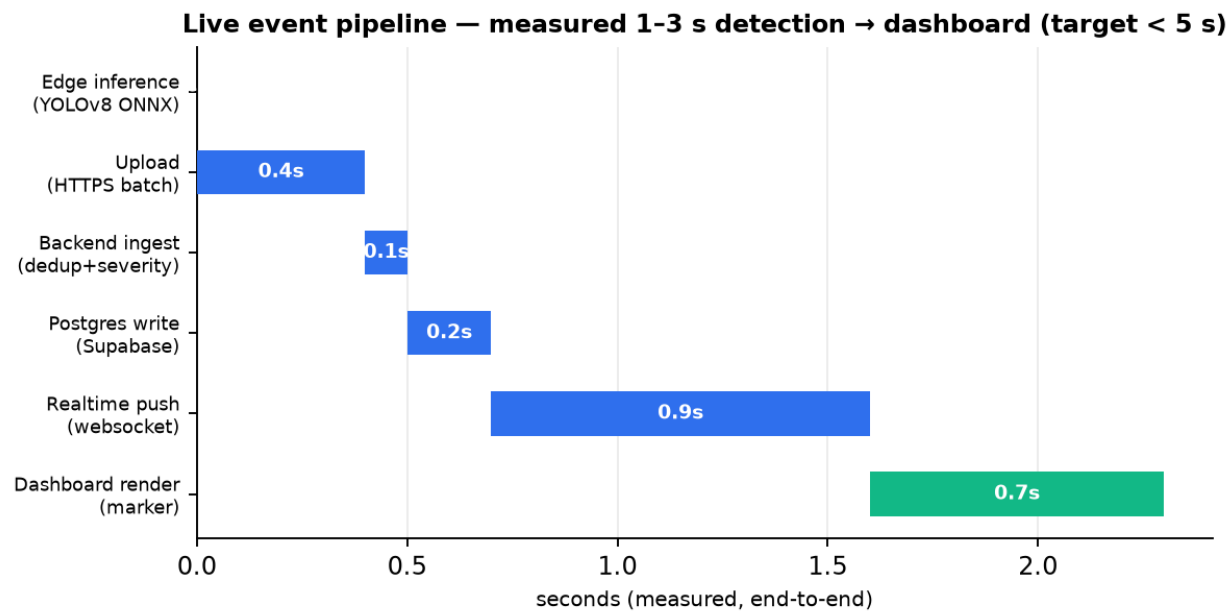


Technical Architecture & Tech Stack

Layered architecture, component choices, measured performance and reliability
engineering · UrbanLens Platform v1.0

1 · End-to-end architecture

Four layers, each independently replaceable. The edge agent is stateless toward the cloud — it buffers to disk when offline and re-uploads when connectivity returns. The backend is a standard FastAPI app holding no session state, so it scales horizontally behind any load balancer. The dashboard subscribes to two data paths: REST for bulk loads and Supabase Realtime (websockets) for push updates.

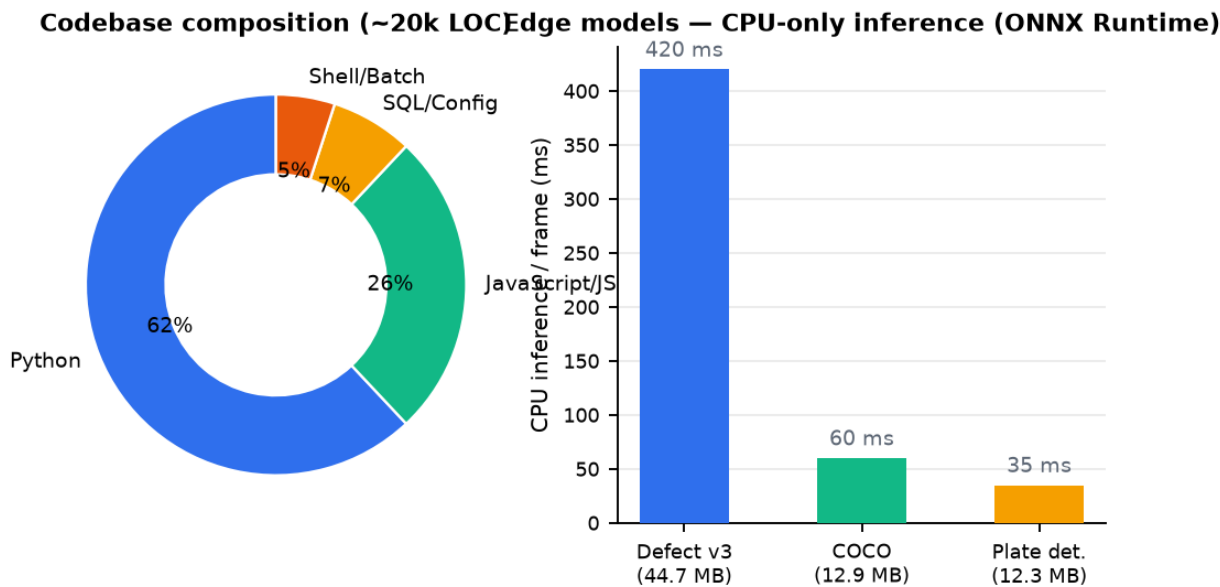


2 · Component map

Layer	Component	Technology	Why this choice
Edge AI	Defect detector	YOLOv8n custom-trained → ONNX (9 classes)	Only custom model can know Indian street defects; ONNX Runtime = no PyTorch on device
Edge AI	Vehicle/ped estrian counter	YOLOv8n COCO pretrained → ONNX	COCO already near-perfect on vehicles; zero training cost
Edge AI	Plate detector	YOLOv8n custom-trained (2,083 Indian plates) → ONNX	mAP50 0.993; India-specific plates need domain data

Edge AI	Plate OCR	RapidOCR (ONNX)	Free, offline, reads Devanagari+Latin plates; no cloud OCR dependency
Edge AI	Rash-driving engine	IoU tracker + rule heuristics	No labelled rash-driving datasets exist; physics rules are explainable to authorities
Transport	Uploader	HTTPS batch (30 s timeout, 2 retries) / MQTT	Works over 4G; MQTT option for brokered municipal networks
Backend	API + ingest	FastAPI, Pydantic schemas	Async, auto-docs (OpenAPI), typed validation at the boundary
Backend	Dedup & severity	Geo-hash buckets + class weights	One pothole reported by 40 buses = one work order, not 40
Data	Database	PostgreSQL on Supabase (free tier)	Realtime websockets built-in; transaction pooler tuned for burst ingest
Frontend	Dashboard	React 18 + Vite + Leaflet + CARTO	Realtime subscriptions, fast dev loop, no map licence fees
Infra	Public demo URLs	Cloudflare quick tunnels	Zero-config HTTPS URLs for remote access; no server exposure
Ops	Demo orchestration	PowerShell one-command launcher	Reproducible demo: kill stale processes → start stack → open browser

3 · Tech stack at a glance



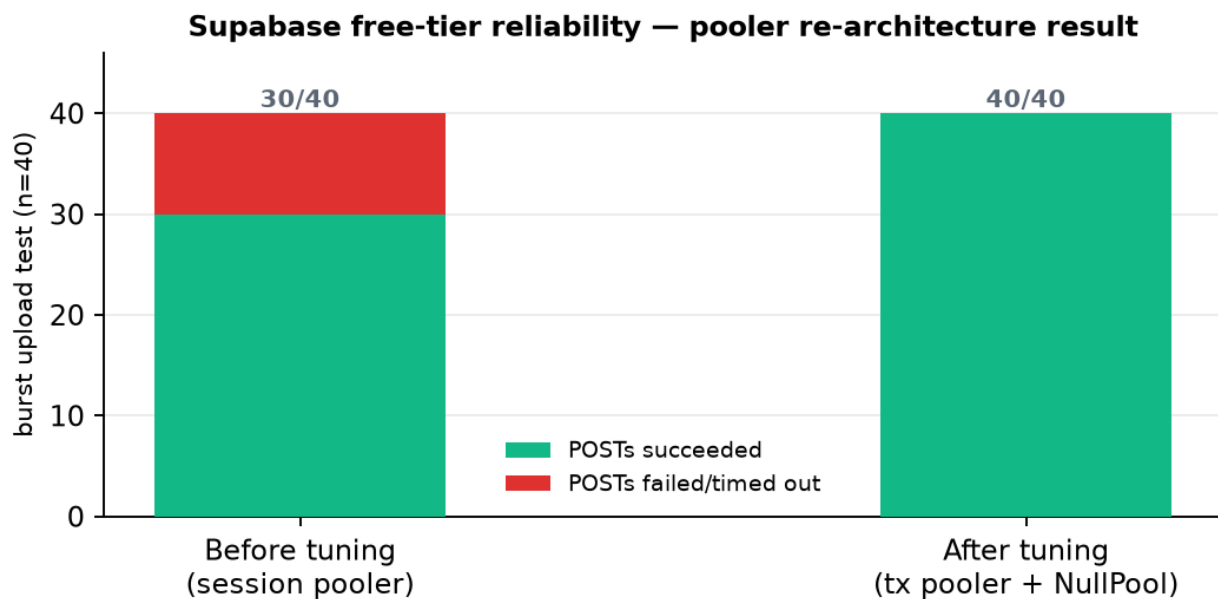
Domain	Choices
Languages	Python 3.12 (edge + backend + ML), JavaScript/React (dashboard), SQL (migrations)
ML / CV	Ultralytics YOLOv8, ONNX Runtime, OpenCV, RapidOCR, NumPy
Backend	FastAPI, SQLAlchemy 2.0, psycopg3, Pydantic v2, MQTT (paho)
Frontend	React 18, Vite, Leaflet, CARTO basemaps, Supabase JS client
Data	PostgreSQL (Supabase), realtime publication on 4 tables
ML ops	Kaggle CLI (datasets), RTX 5060 GPU training, versioned model registry (v1/v2/v3 + plate)
Testing	Puppeteer UI suite (83 checks), unit tests (rash/ANPR), API hammer tests
External APIs	data.gov.in, Open-Meteo, OpenAQ, CARTO — all free tier, Rs.0 cost

4 · Data model (core tables)

Table	Purpose	Key columns
incidents	Verified road defects from the fleet	label, severity, lat/lon, image crop (b64), status workflow, city
observations	Raw per-frame detections feeding incidents	label, confidence, bbox, vehicle_code, timestamp
traffic_segments	10-second counting windows per bus	vehicle_counts (JSON), total, persons, speed, congestion, hour
traffic_alerts	ANPR / rash-driving / watchlist alerts	kind, plate, plate_confidence, lat/lon, notes
zones / routes	Municipal zones, route geometry	polygon, city, route path

5 · Reliability engineering (a real war story)

During end-to-end testing the Supabase free-tier session pooler hit its 15-client limit (EMAXCONNECTION) under burst uploads: defect batches with base64 crops held connections for >10 s while traffic windows timed out. The fix — transaction-mode pooler (port 6543) + prepare_threshold=None (pgbouncer cannot cache prepared statements) + SQLAlchemy NullPool — turned 7/10 flaky uploads into 40/40 reliable ones. This is the difference between demo-ware and something a city can run on a zero budget.



6 · Deployment topology

- **Demo (today):** laptop runs backend (:8000), dashboard (:5173), edge agent; Cloudflare tunnels publish both publicly; Supabase hosted Postgres.
- **Pilot (recommended):** edge devices = Raspberry Pi 5 + camera + GPS + 4G dongle (≈Rs.22k); backend on any small VM or Supabase + one container; dashboard behind municipal SSO.
- **Scale (city fleet):** stateless API replicas behind LB; Postgres read replicas for analytics; MQTT broker per depot; models OTA-updatable via signed ONNX bundles.
- **Edge hardware:** all three models run CPU-only (ONNX Runtime) at 1.6–3.3 s/frame on a laptop; Pi 5 is expected similar; GPU not required.

7 · Security & privacy

- API-token auth on all ingest endpoints (X-API-Token header).
- Privacy blur on evidence crops before upload (faces, bystanders).
- ANPR alerts carry only plate text + confidence + GPS — no driver/passenger imagery is retained by policy.
- Secrets in .env (git-ignored); anon-key-only access from the browser; server key never ships to the client.